

Network Programming & Collection Framework

1. Network Programming in Java

1.1 Introduction to Java Networking

Networking means communication between two or more computers/devices through a network.

In Java, networking can be performed using classes and interfaces provided mainly in the:

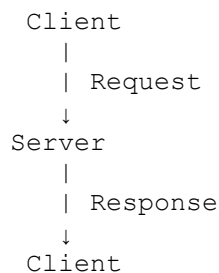
`java.net`

package.

Java networking can be used to:

- Connect two computers
- Send and receive data
- Communicate between client and server
- Access resources over a network
- Create chat applications
- Build client-server applications

Basic Client-Server Model



Client: Requests a service.

Server: Provides the service.

Example

When a browser requests a webpage:

Browser → Web Server
Request

Browser ← Web Server
Response

2. Java Networking Classes

Some commonly used classes in `java.net` are:

Class	Purpose
<code>InetAddress</code>	Represents an IP address
<code>Socket</code>	Creates a connection between client and server
<code>ServerSocket</code>	Used by server to listen for clients
<code>URL</code>	Represents a web address
<code>URLConnection</code>	Establishes connection to a URL
<code>DatagramSocket</code>	Used for UDP communication
<code>DatagramPacket</code>	Represents UDP data packet

For Core Java, the most important classes are:

```
InetAddress
Socket
ServerSocket
```

3. InetAddress Class

`InetAddress` is used to obtain information about a computer's IP address and hostname.

Import:

```
import java.net.InetAddress;
```

Example

```
import java.net.*;

class NetworkDemo {
    public static void main(String[] args) throws Exception {

        InetAddress ip = InetAddress.getLocalHost();

        System.out.println("Host Name: " + ip.getHostName());
        System.out.println("IP Address: " + ip.getHostAddress());
    }
}
```

Possible Output

```
Host Name: Ishan-PC
IP Address: 192.168.1.10
```

The actual output depends on the computer.

Important Methods

Method	Purpose
<code>getLocalHost()</code>	Gets local computer information
<code>getHostName()</code>	Returns hostname
<code>getHostAddress()</code>	Returns IP address
<code>getByName()</code>	Gets IP information for a hostname

Example:

```
InetAddress ip =  
    InetAddress.getByName("google.com");  
  
System.out.println(ip.getHostAddress());
```

4. Socket API

A **socket** is an endpoint used for communication between two computers.

Java provides the:

`Socket`

class for TCP communication.

TCP Communication

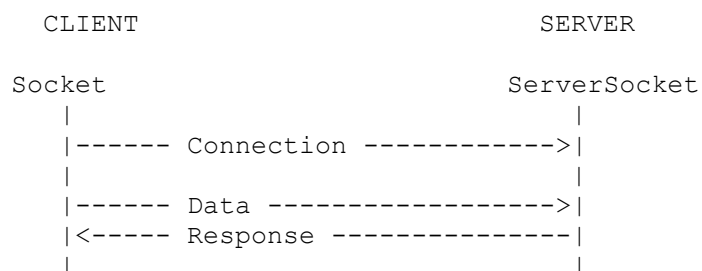
TCP stands for:

Transmission Control Protocol

TCP provides:

- Reliable communication
- Connection-oriented communication
- Ordered data transmission

Client-Server Communication



5. Socket Class

The `Socket` class is used to create a connection between a client and a server.

Import:

```
import java.net.Socket;
```

Basic syntax:

```
Socket s = new Socket("localhost", 5000);
```

Here:

```
localhost → Server address  
5000      → Port number
```

A **port number** identifies a particular application/service running on a computer.

6. ServerSocket Class

`ServerSocket` is used to create a server that waits for client connections.

Example:

```
ServerSocket ss = new ServerSocket(5000);
```

The server waits for a client using:

```
Socket s = ss.accept();
```

`accept()` waits until a client connects.

7. Simple Socket Programming

Server Program

```
import java.net.*;
import java.io.*;

class Server {
    public static void main(String[] args) throws Exception {

        ServerSocket ss = new ServerSocket(5000);

        System.out.println("Server is waiting...");

        Socket s = ss.accept();

        System.out.println("Client connected!");

        ss.close();
    }
}
```

Explanation

```
ServerSocket ss = new ServerSocket(5000);
```

Creates a server on port 5000.

```
Socket s = ss.accept();
```

Waits for a client.

Client Program

```
import java.net.*;

class Client {
    public static void main(String[] args) throws Exception {

        Socket s = new Socket("localhost", 5000);

        System.out.println("Connected to server!");

        s.close();
    }
}
```

How to Run

First run:

```
Server
```

Then run:

```
Client
```

Output:

```
Server:
Server is waiting...
Client connected!
```

```
Client:
Connected to server!
```

8. Sending Data Using Socket

Java sockets use streams to send and receive data.

Important classes:

```
InputStream
OutputStream
```

For text data, we can use:

```
BufferedReader
PrintWriter
```

Client Sends Message to Server

Server

```
import java.net.*;
import java.io.*;

class Server {
    public static void main(String[] args) throws Exception {

        ServerSocket ss = new ServerSocket(5000);

        Socket s = ss.accept();

        BufferedReader br =
            new BufferedReader(
                new InputStreamReader(s.getInputStream()));

        String message = br.readLine();

        System.out.println("Client says: " + message);
    }
}
```

```
        s.close();
        ss.close();
    }
}
```

Client

```
import java.net.*;
import java.io.*;

class Client {
    public static void main(String[] args) throws Exception {

        Socket s = new Socket("localhost", 5000);

        PrintWriter out =
            new PrintWriter(s.getOutputStream(), true);

        out.println("Hello Server!");

        s.close();
    }
}
```

Output

Client:
Connected to server

Server:
Client says: Hello Server!

9. Important Networking Terms

Term	Meaning
IP Address	Address of a computer on a network
Hostname	Name of a computer
Port	Identifies a network service/application
Socket	Communication endpoint
Client	Requests service
Server	Provides service
TCP	Reliable connection-oriented protocol
UDP	Faster connectionless protocol

10. Collection Framework

Introduction

A **Collection** is used to store and manage multiple objects.

For example:

Instead of:

```
String student1 = "Nand";  
String student2 = "Aryan";  
String student3 = "Krisha";
```

we can use a collection:

```
ArrayList<String> students = new ArrayList<>();
```

and store:

```
Nand  
Aryan  
Krisha
```

Why Collections?

Arrays have some limitations:

```
Fixed size  
Difficult insertion/deletion  
Limited built-in operations
```

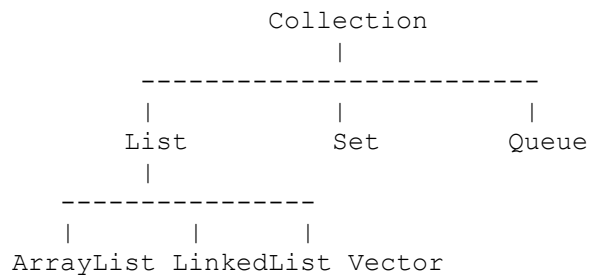
Collections provide:

Dynamic size
Easy insertion
Easy deletion
Searching
Sorting
Many predefined methods

11. Collection Framework

Java provides a framework of interfaces and classes for storing objects.

Basic structure:



For this unit, we mainly focus on:

Collection
List
AbstractList
ArrayList
LinkedList
Enumeration
Vector
Properties

12. Collection Interface

Collection is a major interface in Java's Collection Framework.

It provides common methods for collections.

Some important methods:

Method	Purpose
<code>add()</code>	Adds an element
<code>remove()</code>	Removes an element
<code>contains()</code>	Checks whether element exists
<code>size()</code>	Returns number of elements
<code>clear()</code>	Removes all elements
<code>isEmpty()</code>	Checks whether collection is empty

Example:

```
import java.util.*;

class Demo {
    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("C++");

        System.out.println(list);
        System.out.println(list.size());
    }
}
```

Output:

```
[Java, Python, C++]
3
```

13. List Interface

List is an interface that represents an **ordered collection**.

Important characteristics:

- Maintains insertion order
- Allows duplicate elements
- Allows index-based access
- Allows `null` values

Example:

Index:	0	1	2
	Java	Python	Java

Here "Java" can occur twice.

Common implementations:

```
ArrayList  
LinkedList  
Vector
```

14. ArrayList

ArrayList is one of the most commonly used collection classes.

Import:

```
import java.util.ArrayList;
```

Syntax:

```
ArrayList<String> list =  
    new ArrayList<>();
```

Example

```
import java.util.*;  
  
class ArrayListDemo {  
    public static void main(String[] args) {  
  
        ArrayList<String> names = new ArrayList<>();  
  
        names.add("Nand");  
        names.add("Aryan");  
        names.add("Krisha");  
  
        System.out.println(names);  
    }  
}
```

```
}
```

Output:

```
[Nand, Aryan, Krisha]
```

15. Important ArrayList Methods

add()

```
list.add("Java");
```

Adds an element.

get()

```
System.out.println(list.get(0));
```

Gets an element using index.

set()

```
list.set(0, "Python");
```

Replaces an element.

remove()

```
list.remove(1);
```

Removes the element at index 1.

size()

```
System.out.println(list.size());
```

Returns number of elements.

contains()

```
System.out.println(list.contains("Java"));
```

Returns:

```
true
```

or

```
false
```

16. ArrayList Example

```
import java.util.*;

class StudentList {
    public static void main(String[] args) {

        ArrayList<String> students = new ArrayList<>();

        students.add("Nand");
        students.add("Aryan");
        students.add("Krisha");

        System.out.println("Students: " + students);

        System.out.println("First Student: "
                           + students.get(0));

        students.set(1, "Bhargav");

        System.out.println("After Update: " + students);

        students.remove("Krisha");

        System.out.println("After Remove: " + students);
    }
}
```

Output:

```
Students: [Nand, Aryan, Krisha]
First Student: Nand
After Update: [Nand, Bhargav, Krisha]
After Remove: [Nand, Bhargav]
```

17. LinkedList

LinkedList is another implementation of the List interface.

It stores elements using linked nodes.

[Nand] ↔ [Aryan] ↔ [Krisha]

Syntax:

```
LinkedList<String> list =  
    new LinkedList<>();
```

Example:

```
import java.util.*;  
  
class LinkedListDemo {  
    public static void main(String[] args) {  
  
        LinkedList<String> names = new LinkedList<>();  
  
        names.add("Nand");  
        names.add("Aryan");  
        names.add("Krisha");  
  
        System.out.println(names);  
    }  
}
```

Output:

[Nand, Aryan, Krisha]

18. LinkedList Special Methods

LinkedList provides methods useful for adding/removing elements from the beginning and end.

Add First

```
names.addFirst("Ishan");
```

Add Last

```
names.addLast("Bhargav");
```

Remove First

```
names.removeFirst();
```

Remove Last

```
names.removeLast();
```

Example:

```
LinkedList<String> names = new LinkedList<>();

names.add("Aryan");
names.add("Krisha");

names.addFirst("Nand");
names.addLast("Bhargav");

System.out.println(names);
```

Output:

```
[Nand, Aryan, Krisha, Bhargav]
```

19. ArrayList vs LinkedList

ArrayList	LinkedList
Uses dynamic array internally	Uses linked nodes
Fast random access	Slower random access
Good for accessing elements	Good for frequent insertion/deletion
<code>get()</code> is generally faster	Insert/delete can be efficient
Commonly used	Useful for queue/deque-like operations

Simple Rule

More searching/access → ArrayList

More insertion/deletion → LinkedList

20. AbstractList

`AbstractList` is an **abstract class** in the Java Collection Framework.

It provides a partial implementation of the `List` interface.

Hierarchy:

```
Collection
|
List
|
AbstractList
|
ArrayList
```

It is mainly useful when creating a **custom List implementation**.

Students normally do not create objects directly from `AbstractList` because it is abstract.

```
// Not allowed
AbstractList<String> list =
    new AbstractList<>();
```

Instead:

```
ArrayList<String> list =
    new ArrayList<>();
```

Why AbstractList?

It reduces the amount of code required when a programmer creates a custom list class.

21. Vector

Vector is a dynamic array similar to ArrayList.

Example:

```
import java.util.*;

class VectorDemo {
    public static void main(String[] args) {

        Vector<String> names = new Vector<>();

        names.add("Nand");
        names.add("Aryan");
        names.add("Krisha");

        System.out.println(names);
    }
}
```

Output:

```
[Nand, Aryan, Krisha]
```

Important Point

Vector is an older/legacy collection class.

It is **synchronized**, whereas ArrayList is not.

For most modern single-threaded applications:

```
ArrayList → preferred
Vector → legacy/special cases
```

22. Enumeration

Enumeration is an interface used to traverse elements of some legacy collection classes, especially `Vector`.

Example:

```
import java.util.*;

class EnumerationDemo {
    public static void main(String[] args) {

        Vector<String> names = new Vector<>();

        names.add("Nand");
        names.add("Aryan");
        names.add("Krisha");

        Enumeration<String> e =
            names.elements();

        while (e.hasMoreElements()) {

            System.out.println(e.nextElement());

        }
    }
}
```

Output:

```
Nand
Aryan
Krisha
```

Important Methods

Method	Purpose
<code>hasMoreElements()</code>	Checks whether another element exists
<code>nextElement()</code>	Returns the next element

23. Properties

`Properties` is a special class used to store **key-value pairs**.

It is commonly used for configuration information.

Example:

```
username = ishan
password = java123
database = studentdb
```

Import:

```
import java.util.Properties;
```

Example

```
import java.util.*;

class PropertiesDemo {
    public static void main(String[] args) {

        Properties p = new Properties();

        p.setProperty("name", "Ishan");
        p.setProperty("course", "Java");
        p.setProperty("semester", "5");

        System.out.println(p.getProperty("name"));
        System.out.println(p.getProperty("course"));
        System.out.println(p.getProperty("semester"));
    }
}
```

Output:

```
Ishan
Java
5
```

24. Properties – Important Methods

Method	Purpose
setProperty()	Stores key-value pair
getProperty()	Retrieves value
remove()	Removes property
containsKey()	Checks key
containsValue()	Checks value

Example:

```
p.setProperty("city", "Vadodara");  
  
System.out.println(  
    p.getProperty("city")  
);
```

Output:

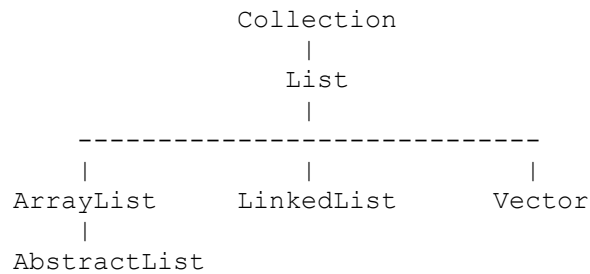
Vadodara

25. Quick Comparison

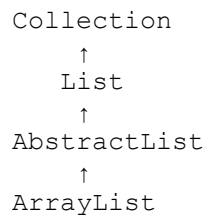
Feature	ArrayList	LinkedList	Vector	Properties
Type	List	List	List	Key-Value
Duplicate values	Yes	Yes	Yes	Keys should be unique
Index-based	Yes	Yes	Yes	No
Dynamic	Yes	Yes	Yes	Yes
Thread-safe/synchronized	No	No	Yes	Yes, legacy synchronization
Main use	General list	Insert/delete	Legacy list	Configuration

26. Important Hierarchy for Students

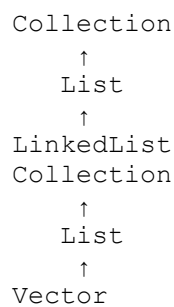
Students should remember this simplified hierarchy:



More accurately:



and:



Enumeration is an **interface used to traverse legacy collections**, not a List class.

Properties is a **separate class** used for key-value configuration data.

27. Important Exam Points

Network Programming

Q. Which package is used for networking in Java?

`java.net`

Q. Which class represents an IP address?

`InetAddress`

Q. Which class is used by a client for TCP communication?

`Socket`

Q. Which class is used by a TCP server?

`ServerSocket`

Q. Which method waits for a client connection?

`accept()`

Q. What is a port?

A port identifies a particular network service/application on a computer.

Collection Framework

Q. Which interface represents an ordered collection?

`List`

Q. Which class provides a dynamic array?

`ArrayList`

Q. Which class uses linked nodes?

`LinkedList`

Q. Which legacy class is synchronized?

`Vector`

Q. Which interface is used to traverse Vector?

`Enumeration`

Q. Which class stores key-value pairs?

Properties

Q. Which method adds an element to an ArrayList?

add()

Q. Which method retrieves an ArrayList element?

get()

Q. Which method returns the number of elements?

size()

One-line Revision

Socket → Client communication
ServerSocket → Server communication
InetAddress → IP/Host information

Collection → Group of objects
List → Ordered collection
ArrayList → Dynamic array
LinkedList → Linked nodes
Vector → Synchronized legacy list
Enumeration → Traverses legacy collections
Properties → Key-value configuration
AbstractList → Partial implementation of List

For your Core Java students, I would teach this in this order:

**Network basics → InetAddress → Socket/ServerSocket → simple client-server program
→ Collection Framework → List → ArrayList → LinkedList → AbstractList → Vector
→ Enumeration → Properties.**